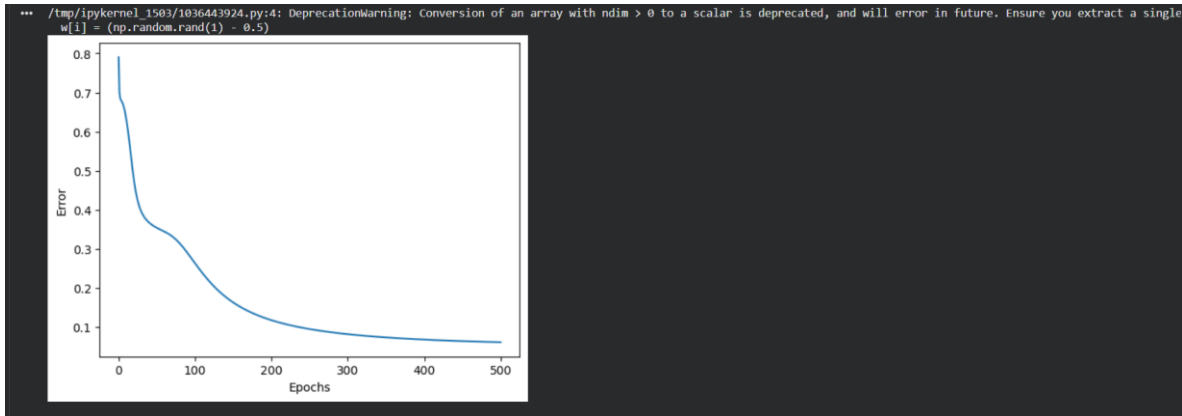


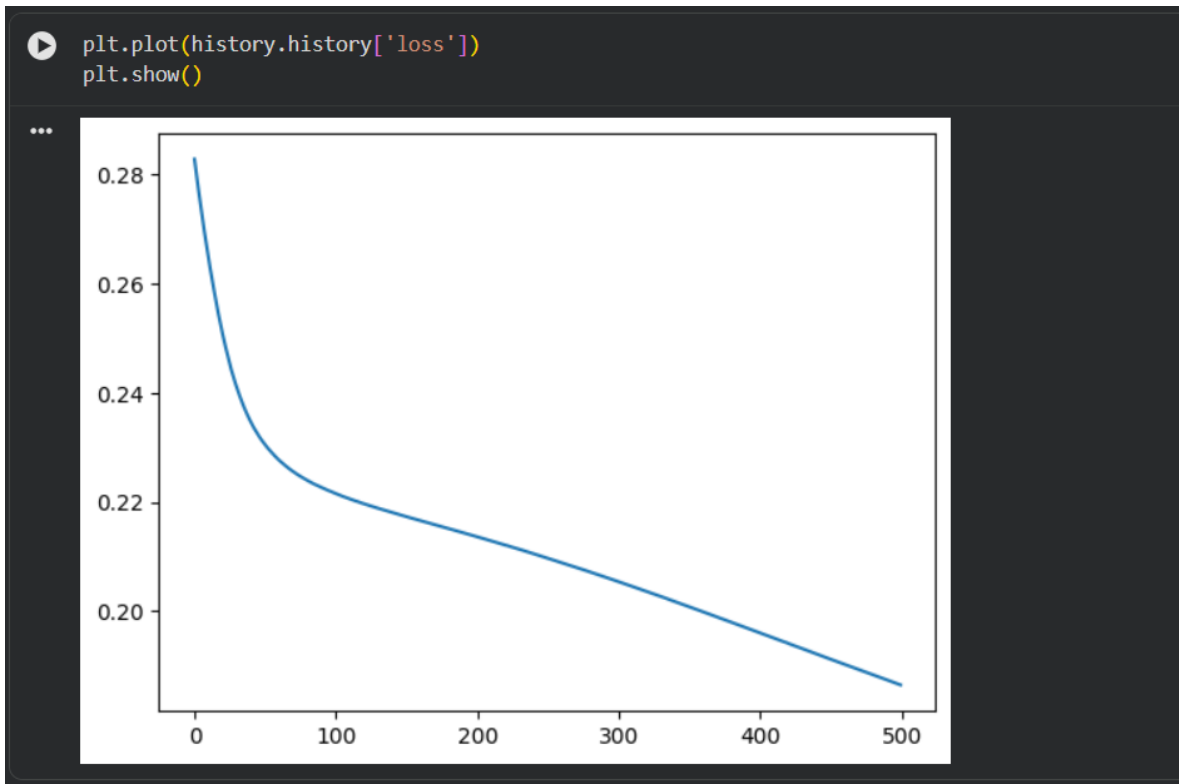
Link Notebook 1: [Notebook01](#)

Link Notebook 2: [Notebook 02](#)

Curva de Error Notebook 1 – versión1



Curva Loss Notebook 2 – Versión 1



Valor de error/pérdida al final de las 500 épocas – versión 1

```
Epoch 496/500
5/5 ————— 0s 7ms/step - loss: 0.1868
Epoch 497/500
5/5 ————— 0s 7ms/step - loss: 0.1867
Epoch 498/500
5/5 ————— 0s 7ms/step - loss: 0.1866
Epoch 499/500
5/5 ————— 0s 7ms/step - loss: 0.1865
Epoch 500/500
5/5 ————— 0s 7ms/step - loss: 0.1865
```

Model Summary y la predicción notebook 2 – versión 1

```
model.predict(np.array([[3,3,1,1]]))
```

```
... 1/1 ————— 0s 37ms/step
array([[0.43531778, 0.37841797, 0.34108478]], dtype=float32)
```

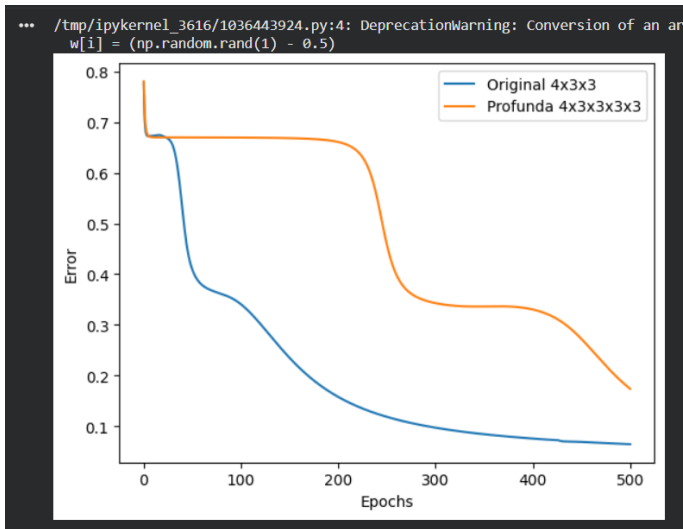
```
model.summary()
```

Model: "sequential"

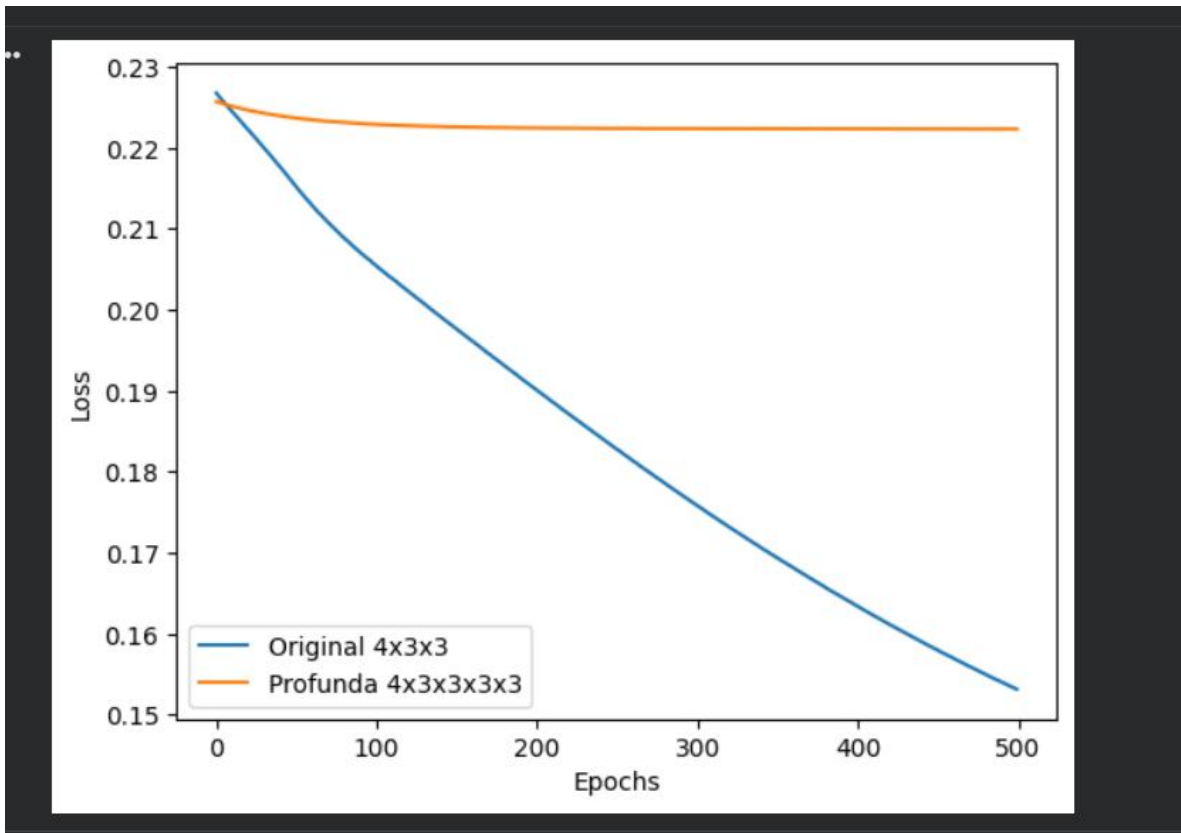
Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12

Total params: 29 (120.00 B)
 Trainable params: 27 (108.00 B)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 2 (12.00 B)

Curva de error notebook 1 – versión 1 vs versión 2



Curva los notebook 2 – versión 1 vs versión 2



Valor de error/pérdida al final de las 500 épocas – versión 1 vs versión 2

```

5/5 ————— 0s 7ms/step - loss: 0.2223
Epoch 491/500
5/5 ————— 0s 7ms/step - loss: 0.2223
Epoch 492/500
5/5 ————— 0s 7ms/step - loss: 0.2223
Epoch 493/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 494/500
5/5 ————— 0s 7ms/step - loss: 0.2223
Epoch 495/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 496/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 497/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 498/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 499/500
5/5 ————— 0s 8ms/step - loss: 0.2223
Epoch 500/500
5/5 ————— 0s 10ms/step - loss: 0.2223

```

Model Summary y la predicción notebook 2 – versión 2

```

model_deep.predict(np.array([[3,3,1,1]]))

```

```

1/1 ————— 0s 69ms/step
array([[0.3340835 , 0.3321319 , 0.33615386]], dtype=float32)

```

```

model_deep.summary()

```

... Model: "sequential_3"

Layer (type)	Output Shape	Param #
layer1 (Dense)	(None, 3)	15
layer2 (Dense)	(None, 3)	12
layer3 (Dense)	(None, 3)	12
layer4 (Dense)	(None, 3)	12

Total params: 53 (216.00 B)
 Trainable params: 51 (204.00 B)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 2 (12.00 B)

Reporte

1. ¿Bajó más el error o se estancó/empeoró?

En las dos implementaciones el error terminó siendo peor con la red profunda. En NumPy pasó de 0.064 a 0.174, y en Keras de 0.153 a 0.222. La forma en que empeoró fue distinta: en NumPy la curva profunda pasa mucho tiempo estancada pero al final sigue bajando, así que con más épocas probablemente seguiría mejorando. En Keras la curva profunda casi no se mueve desde el principio y se queda plana casi todo el entrenamiento, quedando prácticamente estancada. La predicción casi igual para las tres clases confirma que esa red no terminó de aprender.

2. ¿Se parecen las curvas de NumPy y Keras con la misma topología?

Con la red original, las dos curvas bajan de forma parecida, aunque la de Keras es más suave y la de NumPy tiene un tramo inicial donde casi no baja antes de empezar a mejorar. Los números no son exactamente comparables porque cada notebook mide el error de forma distinta: en la notebook 01 se suma el error de las tres salidas, y en Keras se promedia, así que las escalas no coinciden aunque la tendencia sí.

Con la red profunda las curvas se parecen menos, en NumPy sigue bajando hasta el final, en Keras se queda plana casi de inmediato. Esto puede deberse a varias diferencias de implementación.

Los pesos iniciales no se generan igual, la notebook 01 los pone a mano con un rango fijo, y Keras usa su propio método de inicialización por defecto. Un punto de partida distinto puede hacer que la red caiga más o menos rápido en una zona donde le cuesta aprender.

La notebook 01 actualiza los pesos ejemplo por ejemplo y siempre en el mismo orden. Keras, por defecto, agrupa los datos en lotes y los revuelve en cada época. Esa diferencia en cómo se actualizan los pesos puede cambiar qué tan fácil es salir de un estancamiento.

Ninguna de las dos corridas fijó una semilla aleatoria, así que parte de la diferencia puede ser simple casualidad de esa corrida en particular, no una diferencia real entre NumPy y Keras.

3. ¿Tiene sentido que la red más profunda no aprenda mejor?

Sí, tiene sentido. Cuando se entrena con sigmoides, el ajuste de cada capa depende de una derivada que es pequeña cuando la neurona ya está cerca de 0 o de 1, y en el mejor de los casos llega a 0.25. Al agregar más capas de sigmoide antes de la salida,

ese número pequeño se multiplica varias veces para llegar a las primeras capas, así que la señal que le llega a esas capas para corregirse se vuelve todavía más pequeña. Esto es justo lo que se ve en las gráficas: la red profunda pasa mucho tiempo casi sin moverse, mientras que la original baja de forma más constante.

Además, Iris es un problema pequeño y fácil de separar, así que una sola capa oculta ya alcanza para resolverlo bien. Agregar más capas no le da a la red ninguna ventaja real para este problema, pero sí hace más difícil entrenarla. Por eso el resultado tiene sentido: la red profunda terminó peor y, en el caso de Keras, prácticamente sin aprender a distinguir las clases.